INNOMATICS RESEARCH LABS

RAG Internship Project - Final Submission

LOW-LEVEL DESIGN (LLD)

Design & Build a RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Submitted by: Meghna Subramani

Submission Date: April 2026

Deliverable 2 of 3

1. Module-Level Design

1.1 API Layer (api_server.py)

- Handles HTTP requests
- Endpoint: /chat
- Sends query to RAG pipeline
- Returns response in JSON format

1.2 RAG Pipeline (rag_graph.py)

- Core logic of the system
- Built using LangGraph
- Defines step-by-step processing of queries

1.3 Document Loader (document_loader.py)

- Loads PDF files
- Splits text into smaller chunks
- Prepares data for embedding

1.4 Ingestion Script (ingest.py)

- Converts text chunks into embeddings
- Stores embeddings in ChromaDB

1.5 Configuration (config.py)

- Loads environment variables
- Stores settings like model, thresholds, etc.

2. Data Structures

2.1 Graph State Object — SupportState

This TypedDict is the single source of truth passed between all graph nodes:

```
class SupportState(TypedDict):
    query: str                    # Original customer question
    retrieved_docs: List[str]     # Raw top-4 chunks from ChromaDB
    relevant_docs: List[str]      # Chunks surviving relevance grading
    answer: str                   # Final response text
    confidence: float             # 0.0 to 1.0  (0.88 = high, 0.35 = low)
    escalated: bool               # True if HITL was triggered
    escalation_reason: str        # Human-readable reason for escalation
    session_id: str               # Unique conversation identifier
    messages: Annotated[List, operator.add]  # Full conversation history
```

2.2 Document Chunk Format (ChromaDB record)

```
Document(
    page_content = 'Our return policy allows returns within 30 days...',
    metadata = {
        'source': 'data/sample_kb/support_manual.pdf',
        'page':   2,
        'chunk_id': 14
    }
)
```

2.3 API Query-Response Schema

```
# POST /chat  — Request body
{ "query": "What is your return policy?",  "session_id": "abc123" }
```

```
# Response
{ "answer":    "Our return policy allows returns within 30 days...",
  "escalated": false,
  "confidence": 0.88 }
```

2.4 Embedding Structure

Each text chunk is converted into a vector of numbers (384 values). These vectors are stored by ChromaDB internally so they can be searched later.

To find similar text, the system compares these vectors using cosine similarity. This gives a score between 0 and 1, where a higher value means the meanings are more similar.

3. Query Processing Flow (LangGraph)

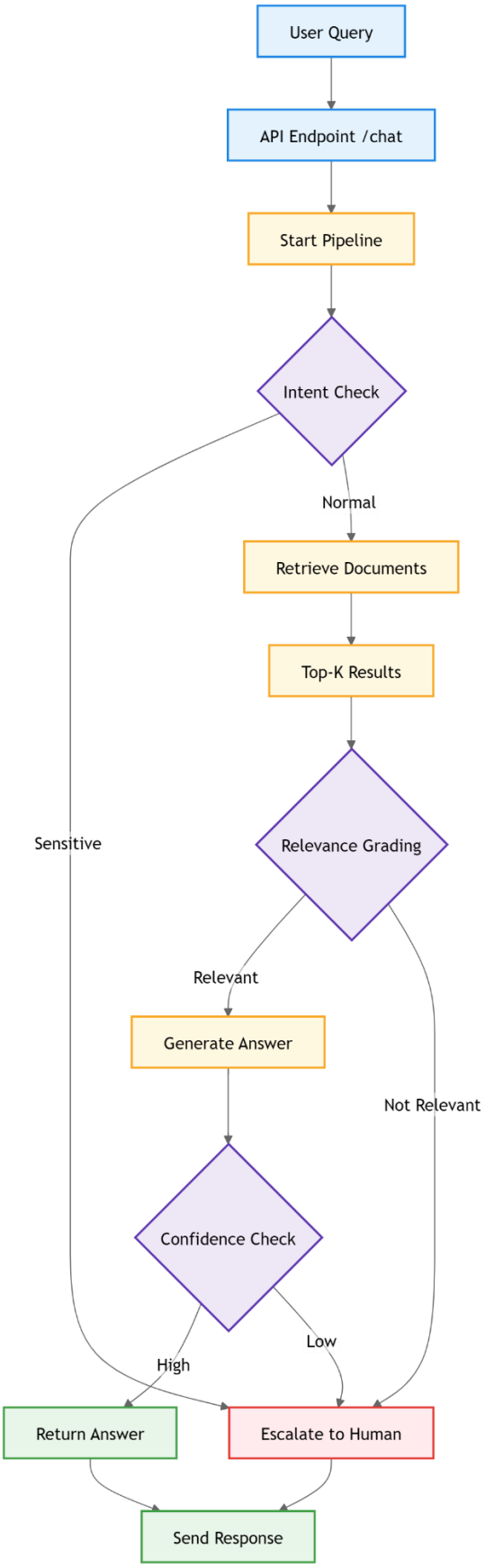
User Input → API → LangGraph → Response

Detailed Steps:

1. User enters query
2. API receives request
3. Query passed to LangGraph pipeline

Inside Pipeline:

4. Intent Check
   - Detect sensitive keywords
5. Retriever
   - Fetch top-K documents from ChromaDB
6. Relevance Grader
   - Filter useful documents
7. Generator
   - Generate answer using LLM
8. Confidence Check
   - Calculate confidence score
9. Escalation
   - If confidence < threshold → mark escalated


User Query
API Endpoint /chat
Start Pipeline
Intent Check
Normal
Sensitive
Retrieve Documents
Top-K Results
Relevance Grading
Relevant
Not Relevant
Generate Answer
Confidence Check
High
Low
Return Answer
Escalate to Human
Send Response

4. Workflow Design (LangGraph)

4.1 Nodes and Responsibilities

Each step in the system is handled by a separate node. Each node takes some input and returns updated information.

Node Name	What it does
Intent Classifier	Checks if the query is sensitive or needs human help
Retriever	Searches the database and gets related documents
Grader	Filters out irrelevant documents
Generator	Creates the final answer using relevant data
HITL Node	Handles cases where human support is needed

4.2 Flow Between Nodes (Routing Logic)

The system decides what to do next based on conditions:

- If the query is sensitive → go directly to HITL
- Otherwise → go to Retriever
- After retrieval → go to Grader
- If no useful documents are found → go to HITL
- If documents are useful → go to Generator
- If confidence is low → go to HITL
- If confidence is high → return answer
- Once HITL is triggered → stop the process

In short:

Query → Check → Retrieve → Filter → Generate → Decide → Answer / Escalate

---

4.3 State Management

The system keeps all information in a shared dictionary called *state*.

- Each node reads the current state
- Updates only what it needs
- Returns the updated values

This makes the system easier to debug and track.
Also, messages are stored in a list, so conversation history keeps growing instead of getting replaced.

5. Human-in-the-Loop (HITL) Design

5.1 When Escalation is Triggered

The system sends the query to a human in three cases:

1. **Sensitive query detected**
   Example: legal issues, complaints
2. **No relevant data found**
   System cannot answer from knowledge base
3. **Low confidence answer**
   Model is unsure about the response

5.2 What Happens After Escalation

When escalation is triggered:

1. The HITL node runs
2. The query and session details are saved
3. The system marks it as escalated
4. The user sees a message like:
   "Your query will be handled by a support agent shortly."
5. The process stops (no further steps run)
6. A human agent later responds through email or support system

---

5.3 How Human Response is Integrated

In the current system:

- Human replies are sent outside the system (like email or CRM)

For a more advanced setup:

- The system can pause at HITL
- Save its current state
- Resume later when a human reply is added

6. API / Interface Design

6.1 Input Format

```
POST /chat
Content-Type: application/json

{ "query":      "What is your return policy?",   // Required. Customer question
  "session_id": "session_abc123"                  // Optional. Defaults to
web_session
}
```

6.2 Output Format

```
HTTP 200 OK
Content-Type: application/json

{  "answer":     "Our return policy allows returns within 30 days of
purchase...",
   "escalated":  false,       // true if routed to HITL
   "confidence": 0.88         // float 0.0-1.0. <0.60 = escalated
}
```

6.3 Interaction Flow

Web UI → POST /chat → FastAPI → run_query() → LangGraph graph.invoke() → all nodes → return state → extract answer/escalated/confidence → JSONResponse → UI renders bubble with badge

CLI → rag_app.py → run_query() → same LangGraph flow → print answer to terminal

6.4 Health Check Endpoint

```
GET /health
Response: { 'status': 'ok' }
```

7. Error Handling

7.1 Error Handling

- If API fails → return error message
- If no documents found → fallback response
- If LLM fails → return safe message

7.2 Testing

- Tested using CLI and Web UI
- Verified:
  - Retrieval accuracy
  - Response quality
  - Escalation behaviour

7.3 Challenges Faced

- Debugging LangGraph pipeline was initially confusing
- Handling empty retrieval results
- Managing API keys and environment setup
- Tuning confidence threshold

7.4 Improvements Done

- Added relevance grading step
- Improved chunking strategy
- Tuned TOP_K value
- Added escalation mechanism